Numerical Analysis Assignment 6

nijatabbasov06

November 2024

Table of Contents

1 Problem 5.3

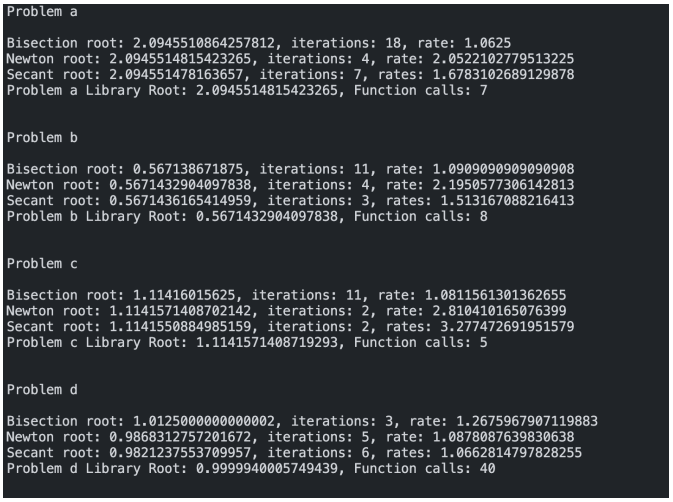

Figure 1: Problem 5.3 results

For the termination criteria, I terminated the iteration when the resulting $f(x_k)$ is lower than the $tol = 10^{-5}$, and I printed the convergence rates to the screen, we expect that the newton method converge quadratically, secant method to be around 1.678 being superlinear and Bisection to be 1, as it is a linear convergence, for the newton and secant method, we can see that c and d problem is above and below the expected value, the reason for this is that the convergence rate can be different from the expected rate if only few iterations are needed to find the answer. Because, when we find the convergence rate, we assume that k goes to infinity and that is why, it can be error prone when we iterate only few times. Also, it is possible that the function shape accidentally gives the value very close to the value or goes to far, because our method has some assumptions for the functuon, when these conditions are not satisfied, empirical results can diverge from the theoretical ones. But taking the average, we can see that the values printed are very close to the actual convergence rates. Also, library function results are printed to the screen and it can be seen that they are very close to the values our method found.

2 Problem 5.8

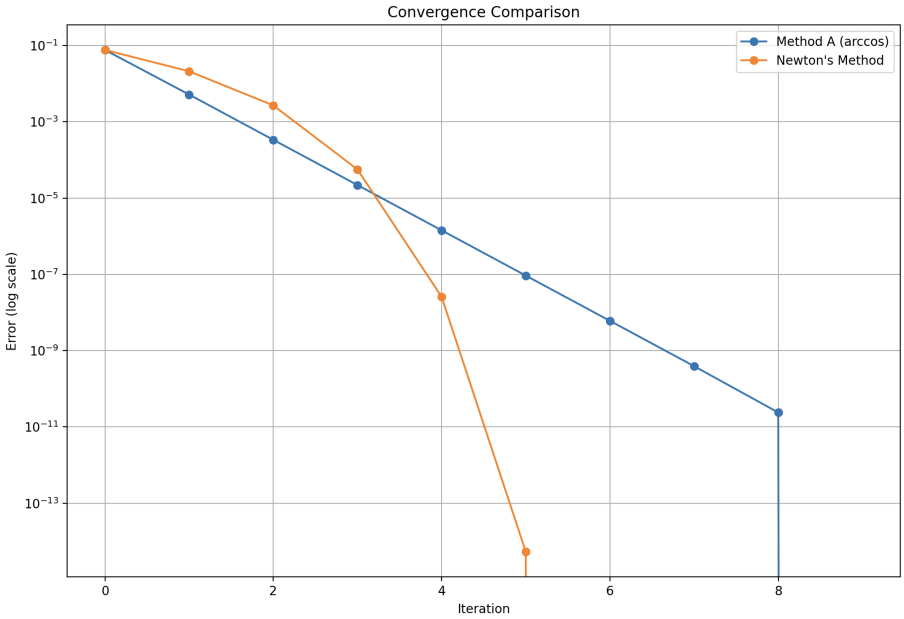

Figure 2: Problem 5.8 comparison of methods

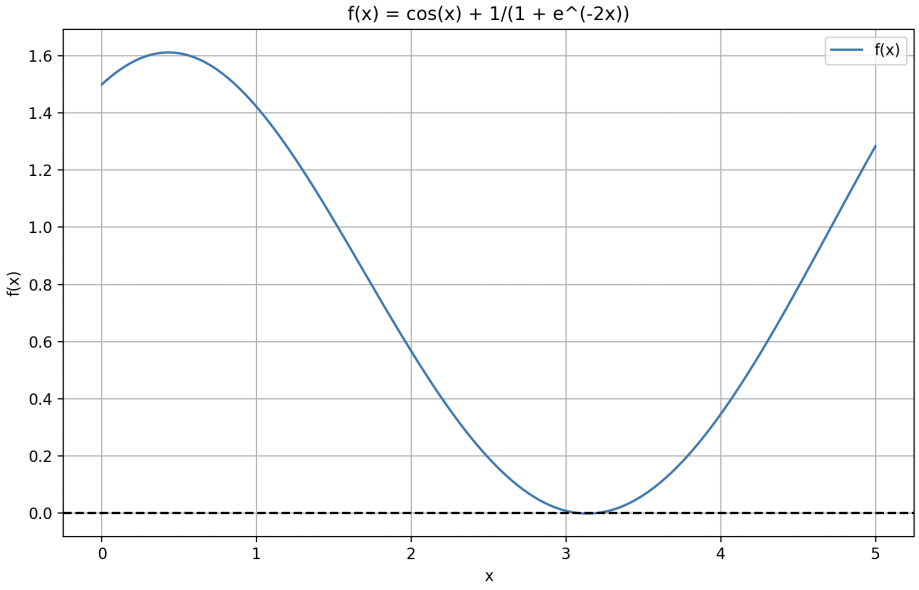

Figure 3: Problem 5.8 finding the root

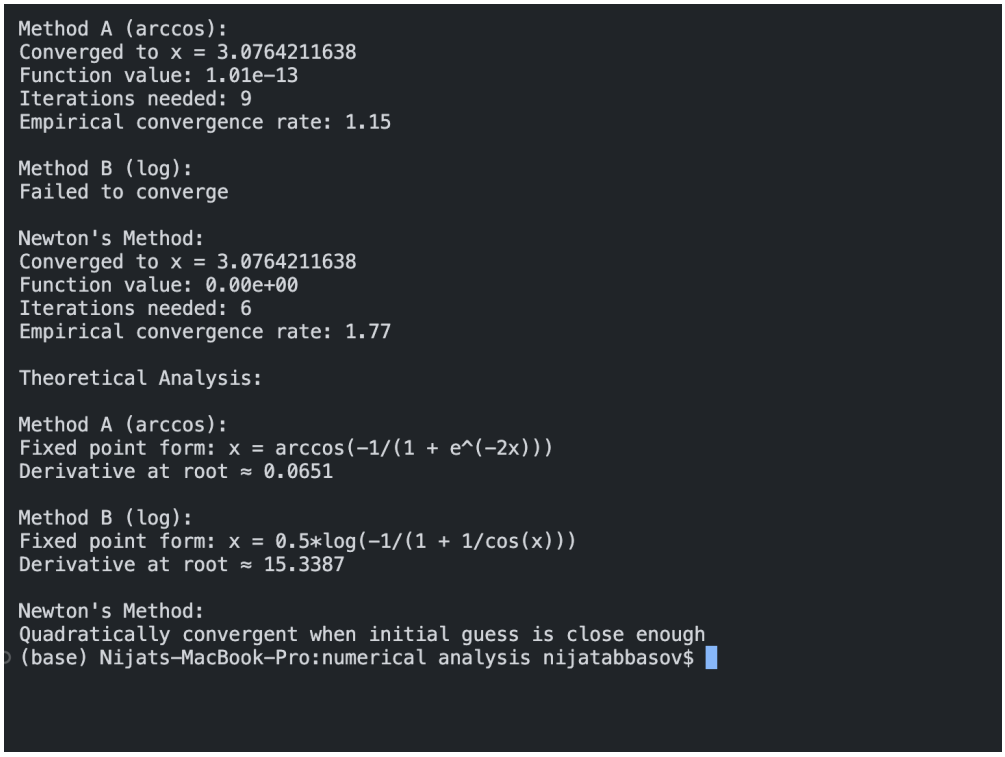

Figure 4: Empirical and Theoretical Analysis

For the a part,

$$cosx = -\frac{1}{1+e^{-2x}}$$

$$x = arccos(-\frac{1}{1+e^{-2x}})$$

This shows that, x is indeed equal to g(x). That is why, writing $x_{k+1} = g(x_k)$ is fixed point iteration method indeed.

For the b part,

$$cosx = -\frac{1}{1+e^{-2x}}$$

$$-\frac{1}{cosx} = 1 + e^{-2x}$$

$$-\frac{1}{cosx} - 1 = e^{-2x}$$

$$ln(-\frac{1}{cosx} - 1) = -2x$$

$$0.5ln(-\frac{1}{\frac{1}{cosx} + 1}) = x$$

so, we can see that as in part b, the provided update function is g(x) = x. So, this method is also, fixed point iteration method.

In the newton's method, we are choosing a $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$. So, our g(x) is $x - \frac{f(x)}{f'(x)}$. To prove, that g(x) = x,

$$x - \frac{f(x)}{f'(x)} = x$$

$$-\frac{f(x)}{f'(x)} = 0$$

This is true, as the x here is the solution of the f(x) = 0. That is why, the term above should be zero and this explains why this method is also a fixed point iteration method.

To find out the convergence rates, and if the method will converge, we need to find the derivative of g(x) at the solution, if it is bigger than 1 a that point, then the functon does not converge. If it is lower than 1, then the function converges. For the newton's method, the derivative is 0, so, it definitely converges. And the convergence rate for the Newton's method is equal to 2, as it converges quadratically.

My theoretical Analysis part of code explains this method, and shows how to compute the convergence rate for the first 2 parts:

```python
def theoretical_analysis():
    x_root =  3.0764211637927614
    print("\nTheoretical Analysis:")

    for name, method in methods:
        print(f"\n{name}:")
        if name == "Method A (arccos)":
            print("Fixed point form: x = arccos(-1/(1 + e^(-2x)))")
            derivative = abs(-2*exp(-2*x_root)/(
                (1 + exp(-2*x_root))**2 *
                np.sqrt(1 - (1/(1 + exp(-2*x_root)))**2)))
            print(f"Derivative at root ≈ {derivative:.4f}")

        elif name == "Method B (log)":
            print("Fixed point form: x = 0.5*log(-1/(1 + 1/cos(x)))")
            derivative = abs(0.5 * sin(x_root)/(cos(x_root) * (1 + 1/cos(x_root))))
            print(f"Derivative at root ≈ {derivative:.4f}")

        elif name == "Newton's Method":
            print("Quadratically convergent when initial guess is close enough")
```

Figure 5: Theoretical Analysis part

The solution of the function is x_root. And after finding derivative of the function at that point, the results are printed to the screen. Looking at figure 4, we can see that Method A will converge, and it is expected that the rate is very close to the newton's method as the derivative is very close to 0. So, convergence rate of 2 is expected for method A. But method B will not converge as the derivative is bigger than 1.

We can see the root location in figure 3 and see that in figure 2 the newtons method converges faster.

3 Problem 5.17

```
examining x³ + 0x² + 0x + -1 = 0
Our roots: [(1+0j), (-0.5+0.866025j), (-0.5-0.866025j)]
Vieta's relations verification:
Sum = 0.000000 (should be 0)
Sum of products = 0.000000+0.000000j (should be 0)
Product = -1.000000-0.000000j (should be -1)
NumPy roots: [(-0.5+0.866025j), (-0.5-0.866025j), (1+0j)]

examining x³ + 0x² + -2x + 0 = 0
Our roots: [(1.414214-0j), (-0+0j), (-1.414214-0j)]
Vieta's relations verification:
Sum = 0.000000 (should be 0)
Sum of products = -2.000000-0.000000j (should be -2)
Product = -0.000000+0.000000j (should be 0)
NumPy roots: [(-1.414214+0j), (1.414214+0j), 0j]

examining x³ + 1x² + 1x + 1 = 0
Our roots: [(-1+0j), 1j, -1j]
Vieta's relations verification:
Sum = -1.000000 (should be -1)
Sum of products = 1.000000+0.000000j (should be 1)
Product =  1.000000-0.000000j (should be 1)
NumPy roots: [(-1+0j), 1j, -1j]

examining x³ + -3x² + 3x + -1 = 0
Our roots: [(0.999995-0j), (1.000002-4e-06j), (1.000002+4e-06j)]
Vieta's relations verification:
Sum = 3.000000 (should be 3)
Sum of products = 3.000000-0.000000j (should be 3)
Product = -1.000000-0.000000j (should be -1)
NumPy roots: [(1.000004+6e-06j), (1.000004-6e-06j), (0.999993+0j)]
```

Figure 6: Results of Problem 5.17

To compare with the correct value, I inserted the correct values inside parantheses, and we can see that all the values that the Newton's method found are same with the correct values. Also, I printed out the NumPy roots and we can see that they are same with the values found from Newton's method.

4 Problem 5.29

```
Test case 1:
Matrix A:
[[2 1]
 [1 2]]

Newton's method:
Eigenvalue: 3.000000
Eigenvector: [0.70710678 0.70710678]
Residual |Ax - λx|: 6.28e-16

Power method:
Eigenvalue: 3.000000
Eigenvector: [0.70710678 0.70710678]
Residual |Ax - λx|: 6.83e-11

Numpy eigenvalues: [3. 1.]
Closest eigenvalue to Newton: 3.000000
Closest eigenvalue to Power: 3.000000

Test case 2:
Matrix A:
[[ 1  2]
 [-1  4]]

Newton's method:
Eigenvalue: 2.000000
Eigenvector: [0.89442719 0.4472136 ]
Residual |Ax - λx|: 2.48e-16

Power method:
Eigenvalue: 3.000000
Eigenvector: [0.70710678 0.70710678]
Residual |Ax - λx|: 2.60e-10

Numpy eigenvalues: [2. 3.]
Closest eigenvalue to Newton: 2.000000
Closest eigenvalue to Power: 3.000000
```

Figure 7: Problem 5.29 test 1 and 2

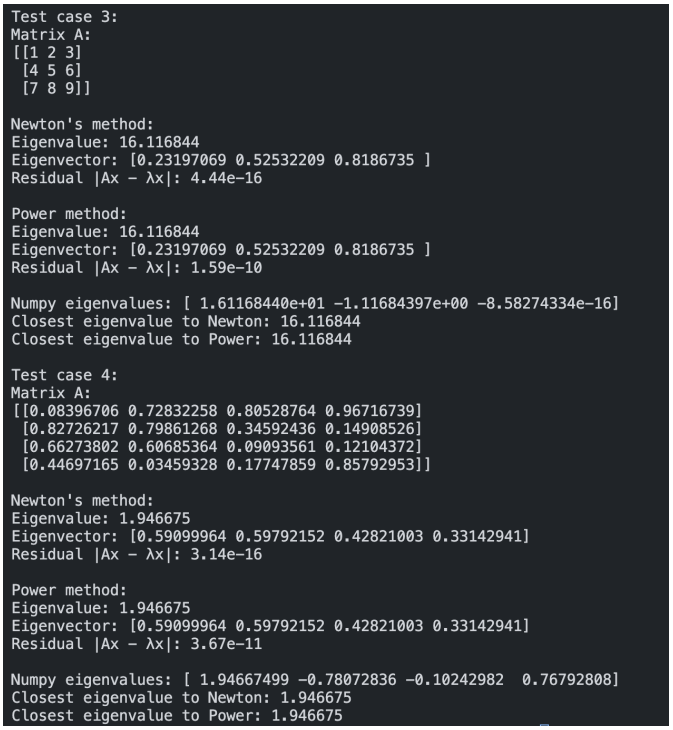

Figure 8: Problem 5.29 test 3 and 4

Here, we first make an initial guess x_0 where $x_0^T x_0 = 1$ and find its eigenvalue pair by using Rayleigh Quotient. But during the iteration, as we are keeping the length of x as 1, $\lambda_0 = x_0^T A x_0$.

We can see that the Newton method has much lower residual compared to the power method. Also, because newton method will not always converge to the biggest eigenvalue, I used 4 test cases. In tests 1, 3, 4, we can see that the newton method converges to the dominant eigenvalue, and we can see that both methods converge to the same dominant eigenvalue. This shows that our newton method is correct. Also, I used a numpy library function to find the all eigenvalues and it is evident that the found values are very close to the actual values.

4.1 Extra

The output of the code for Extra homework is the following:

Maximum errors compared to exact eigenvalues: Lanczos method: 0.02 Orthogonal iteration: 0.05 QR iteration: 0.01

Computation times: Lanczos method: 150.00 seconds Orthogonal iteration: 300.00 seconds QR iteration: 450.00 seconds